Week 4, Day 5: Project Day, Wiring Kafka and Redis End to End, and Testing

Prepared for Upendra Kumar Sharma | 4 to 5 hours | Goal: a working order saga across services, Redis caching and rate limiting, and a real test suite with JUnit, Mockito and Testcontainers

Today's Schedule

- **0:00 to 0:20** Review the design and the definition of done (sections 1 and 2).
- **0:20 to 1:40** Wire the saga: inventory consumer, order consumer, notification consumer (section 3).
- **1:40 to 2:10** Add Redis caching and rate limiting (section 4).
- **2:10 to 3:40** Write the tests (sections 5 and 6).
- **3:40 to 4:10** Run the failure drills (section 7).
- **4:10 to 4:40** Review the talking points and the checklist (sections 8 and 10). Optional light DSA revision (section 9).

No new theory today. The learning comes from making the pieces work together and proving that they do.

1. The Flow You Are Completing

1. **order-service** creates the order as PENDING and writes an OrderCreated row to its outbox in one transaction. The relay publishes it to **orders.created** with the order id as the key.
2. **inventory-service** consumes orders.created. In one database transaction it records the event id (inbox), reserves stock for all items with atomic conditional updates, and writes **InventoryReserved** to its own outbox. If any item cannot be reserved, the whole reservation rolls back and a separate transaction records **InventoryRejected**.
3. **order-service** consumes the result and sets the order to **CONFIRMED** or **CANCELLED**, only if it is still PENDING (a conditional update, so duplicates are harmless).
4. **notification-service** consumes the order outcome events and sends a notification (a log line or a local mail catcher).

Topics (3 partitions each for development): orders.created, inventory.reserved, inventory.rejected, orders.confirmed, orders.cancelled. Create them with NewTopic beans (TopicBuilder.name(...).partitions(3).replicas(1).build()) so the setup is repeatable.

Event payloads: each carries eventId, type, version, occurredAt, orderId and the data consumers need. Consumers ignore unknown fields (a tolerant reader).

Decision: no shared event library. Each service defines **its own small DTO records** for the events it reads. Sharing one jar couples release cycles. Consume the payload as a **String** and parse it with Jackson into the local record, which also avoids Java type headers in messages. Say this in interviews as a deliberate choice, and mention that a Schema Registry would be the next step.

2. Definition of Done for Today

- Placing an order through the gateway ends in CONFIRMED when stock exists, and CANCELLED when it does not.
- Delivering any message twice never changes the outcome twice.
- Stock reads are cached in Redis, and order creation is rate limited per user.
- Each service has unit tests and integration tests that run with one command, and no test depends on your local machine's state.

3. Wiring the Saga

Common consumer configuration (each service)

```
spring:
  kafka:
    bootstrap-servers: kafka:19092
    consumer:
      auto-offset-reset: earliest
      enable-auto-commit: false
      key-deserializer:
org.apache.kafka.common.serialization.StringDeserializer
      value-deserializer:
org.apache.kafka.common.serialization.StringDeserializer
    listener:
      ack-mode: record
      concurrency: 3
```

Add the **DefaultErrorHandler with a dead letter recoverer** from Week 4, Day 2, and mark parsing and validation errors as not retryable.

inventory-service: the listener

```
@Component
public class OrderCreatedListener {
    private final ObjectMapper mapper;
    private final ReservationService reservations;
    // constructor omitted

    @KafkaListener(topics = "orders.created", groupId = "inventory-service")
    public void on(ConsumerRecord<String, String> record) throws
JsonProcessingException {
        OrderCreated event = mapper.readValue(record.value(),
OrderCreated.class);
        reservations.handle(event);
    }
}

public record OrderCreated(String eventId, String orderId, String customerId,
List<Item> items) {
    public record Item(String sku, int quantity) {}
}
```

inventory-service: all-or-nothing reservation with an inbox and an outbox

```
@Service
public class ReservationService {
    private final TransactionTemplate tx;
    private final ProcessedMessageRepository processed;   // INSERT ... ON
CONFLICT DO NOTHING
    private final StockRepository stock;                  // conditional
decrement
    private final OutboxRepository outbox;
    // constructor omitted

    public void handle(OrderCreated event) {
        try {
            tx.executeWithoutResult(status -> reserveAll(event));
        } catch (InsufficientStockException ex) {
            tx.executeWithoutResult(status -> reject(event, ex.getSku()));
// new transaction
        }
    }

    private void reserveAll(OrderCreated event) {
        if (processed.insertIfNew(event.eventId()) == 0) return;      //
duplicate: do nothing
        for (OrderCreated.Item item : event.items()) {
            int updated = stock.decrementIfAvailable(item.sku(),
```

```
item.quantity());
            if (updated == 0) throw new
InsufficientStockException(item.sku());   // rolls everything back
        }
        outbox.save(OutboxEvent.of("Order", event.orderId(),
"InventoryReserved", reservedJson(event)));
    }

    private void reject(OrderCreated event, String sku) {
        if (processed.insertIfNew(event.eventId()) == 0) return;
        outbox.save(OutboxEvent.of("Order", event.orderId(),
"InventoryRejected", rejectedJson(event, sku)));
    }
}
```

- The failed first transaction rolls back **both** the dedupe row and any partial stock changes, so nothing is half done. The second transaction records the rejection exactly once.
- An unknown sku also gives an update count of zero, and takes the same rejection path.
- inventory-service needs its **own outbox relay** (the same polling publisher as order-service, publishing to inventory.reserved and inventory.rejected with the order id as the key).

order-service: apply the outcome idempotently

```
@Transactional
public void onInventoryOutcome(String eventId, UUID orderId, boolean
reserved) {
    if (processed.insertIfNew(eventId) == 0) return;
    String newStatus = reserved ? "CONFIRMED" : "CANCELLED";
    int changed = orders.transition(orderId, "PENDING", newStatus);   //
UPDATE ... WHERE id = ? AND status = 'PENDING'
    if (changed == 1) {
        outbox.save(OutboxEvent.of("Order", orderId.toString(),
                reserved ? "OrderConfirmed" : "OrderCancelled",
outcomeJson(orderId)));
    }
}
```

- Listen on inventory.reserved and inventory.rejected, with the same String-and-parse approach, and add order-service's inbox table in a new Flyway migration.
- The conditional update means a late or duplicated event cannot move an order out of its final state.

notification-service: consume orders.confirmed and orders.cancelled and log (or send) a message. Use the inbox here too, because a notification sent twice is a visible bug.

Checkpoint: create an order for a sku with stock and watch the status change to CONFIRMED within seconds. Create one for more than the stock and watch CANCELLED, with the stock unchanged.

4. Redis: Caching and Rate Limiting

- **Dependencies and config:** spring-boot-starter-data-redis (and the reactive starter for the gateway), with spring.data.redis.host set to redis, and short timeouts (500 ms).
- **Cache the stock read** in inventory-service with the cache-aside code or @Cacheable (Week 4, Days 3 and 4). Evict (or let the short TTL expire) when stock changes, **after commit**. **Never use the cached value to decide a reservation.** The atomic database update decides.
- **Rate limit order creation:** use the gateway RequestRateLimiter (token bucket) with a KeyResolver based on the user id from the token, or your own Lua limiter in order-service. Return 429 with Retry-After.
- **Fail open with a log and alert** if Redis is unavailable, and confirm the API still works with Redis stopped.
- Add the Redis environment variables to the Compose services that use it (SPRING_DATA_REDIS_HOST set to redis).

5. The Testing Strategy

The pyramid: many fast **unit tests**, a good number of **slice and integration tests** against real infrastructure, and a few **end-to-end smoke checks** with Docker Compose.

- **Unit tests (JUnit 5, Mockito, AssertJ):** business rules, state transitions and edge cases, with no Spring and no containers. They run in milliseconds.
- **Slice tests:** @WebMvcTest for controllers (validation, status codes, security), @DataJpaTest with a real PostgreSQL container for repositories and migrations.
- **Integration tests (Testcontainers):** start real PostgreSQL, Kafka and Redis in Docker for the test run. They catch problems mocks never will: SQL syntax, migrations, serialization, consumer behavior, transactions.
- **Why not H2 and embedded fakes?** They differ from PostgreSQL in SQL dialect and locking, so tests can pass while production fails. Testcontainers gives you the same engine as production.
- **Async assertions:** use **Awaitility**, never Thread.sleep. Wait until a condition becomes true, with a timeout.

- **Speed:** declare containers as static fields (or a shared base class) so they start once per test class or per run, and reuse the Spring context where possible.

What to test, specific to this project

- **Idempotency:** the same POST with the same Idempotency-Key returns the same order. The same Kafka event delivered twice changes stock once.
- **Atomicity:** if saving the outbox row fails, no order row exists.
- **Saga paths:** the happy path (reserved), the rejection path (insufficient stock, nothing changed), and a duplicate of each.
- **Concurrency:** many parallel reservations for limited stock never oversell.
- **Security:** no token gives 401, a wrong role gives 403, and a user cannot read someone else's order.
- **Cache and limiter:** the second read does not hit the database, and request number N plus 1 gets a 429.

6. Test Code to Adapt

Dependencies: spring-boot-starter-test, spring-security-test, spring-kafka-test, the Testcontainers modules (junit-jupiter, postgresql, kafka) and awaitility.

Unit test with Mockito: the idempotency key

```
@ExtendWith(MockitoExtension.class)
class OrderServiceTest {
    @Mock OrderRepository orders;
    @Mock OutboxRepository outbox;
    @Mock IdempotencyKeyRepository keys;
    @InjectMocks OrderService service;

    @Test
    void returnsTheExistingOrderWhenTheKeyWasAlreadyUsed() {
        UUID id = UUID.randomUUID();
        Order existing = Order.pending(id, "alice", List.of());
        when(keys.findById("key-1")).thenReturn(Optional.of(new
IdempotencyKey("key-1", id)));
        when(orders.findById(id)).thenReturn(Optional.of(existing));

        OrderResponse response = service.placeOrder("alice", "key-1",
validRequest());

        assertThat(response.id()).isEqualTo(id);
```

```
        verify(orders, never()).save(any());
        verify(outbox, never()).save(any());
    }
}
```

Controller test: security and validation

```
@WebMvcTest(OrderController.class)
class OrderControllerTest {
    @Autowired MockMvc mvc;
    @MockBean OrderService service;      // @MockitoBean in newer Spring Boot
versions

    @Test
    void rejectsRequestsWithoutAToken() throws Exception {
        mvc.perform(get("/api/orders")).andExpect(status().isUnauthorized());
    }

    @Test
    void rejectsAnInvalidBody() throws Exception {
        mvc.perform(post("/api/orders")
                .with(jwt().authorities(new
SimpleGrantedAuthority("ROLE_USER")))
                .contentType(MediaType.APPLICATION_JSON)
                .content("{\"items\":[{\"sku\":\"SKU-1\",\"quantity\":0}]}"))
           .andExpect(status().isBadRequest());
    }
}
```

Integration test with Testcontainers (Spring Boot 3.1 and later can wire the connections for you)

```
@SpringBootTest
@Testcontainers
class ReservationIT {

    @Container @ServiceConnection
    static PostgreSQLContainer<?> postgres = new PostgreSQLContainer<>
("postgres:16");

    @Container @ServiceConnection
    static KafkaContainer kafka = new
KafkaContainer(DockerImageName.parse("apache/kafka:3.8.0"));

    @Container @ServiceConnection(name = "redis")
    static GenericContainer<?> redis = new GenericContainer<>
```

```
("redis:7").withExposedPorts(6379);

    @Autowired KafkaTemplate<String, String> kafkaTemplate;
    @Autowired StockRepository stock;

    @Test
    void deliveringTheSameEventTwiceReservesStockOnce() {
        stock.save(new Stock("SKU-1", 10));
        String event = orderCreatedJson("evt-1", "order-1", "SKU-1", 3);

        kafkaTemplate.send("orders.created", "order-1", event);
        kafkaTemplate.send("orders.created", "order-1", event);      //
duplicate

        await().atMost(Duration.ofSeconds(15)).untilAsserted(() ->
            assertThat(stock.findById("SKU-
1").orElseThrow().getAvailable()).isEqualTo(7));
    }
}
```

- Check the Testcontainers documentation for the Kafka container class that matches your image, since the class and its package changed between releases.
- To prove the second event was processed and ignored (not just late), also assert on the processed_message table count, or wait for both records to be consumed before asserting.
- For the atomicity test, use a spy bean on the outbox repository that throws, call placeOrder, and assert that the orders table is empty (@SpyBean in older Boot versions, @MockitoSpyBean in newer ones).
- For the oversell test, start 50 threads reserving 1 unit each against stock of 10, wait for them to finish, and assert exactly 10 succeeded and stock is 0.

Habits that keep tests healthy

- Name tests as behavior (returnsTheExistingOrderWhenTheKeyWasAlreadyUsed). Use unique ids per test and clean tables in a @BeforeEach so tests do not depend on each other.
- Avoid over-mocking. If a test verifies only that mocks were called, it protects nothing.
- Fix flaky tests at once. A flaky suite gets ignored, which is worse than a small suite.
- Run everything with one command (mvn verify) and measure coverage with JaCoCo as a guide, not a goal.

7. Failure Drills (Run Them by Hand)

With the whole stack running in Compose, break things and watch the system recover. Write the results in your README.

1. **Kafka down when an order is placed.** The order and outbox row commit. When Kafka returns, the relay publishes and the saga completes.
2. **Crash after commit, before publish.** Stop order-service right after the order is saved. On restart the relay publishes the pending outbox row.
3. **Consumer crash before acknowledgement.** Kill inventory-service mid-processing. On restart the event is redelivered, and stock is reserved once.
4. **Duplicate event.** Re-send an old event from the console producer. Nothing changes.
5. **Poison message.** Send invalid JSON to orders.created and confirm it reaches the dead letter topic without blocking later messages.
6. **Inventory service down.** The order stays PENDING. Optional improvement: a scheduled sweeper that cancels or flags orders pending longer than a set time.
7. **Redis down.** The API still works with higher latency, and the limiter fails open with a logged warning.

8. Interview Talking Points

1. How do you test an event-driven system? Mention Testcontainers, Awaitility, idempotency tests and failure drills.
2. Why real containers instead of mocks or H2? Give a concrete example of what mocks miss.
3. How do you prove a consumer is idempotent? Deliver the same event twice and assert one effect.
4. How do you prove outbox atomicity? Force a failure after the business insert and assert both rows are absent.
5. How do you test for overselling under concurrency?
6. What is your test pyramid, and what runs in CI?
7. How did you handle a partial reservation across several items?
8. Why does each service own its event DTOs? What would you change with a Schema Registry?
9. What happens to an order if inventory-service is down for ten minutes?
10. Which part of this project failed during a drill, and what did you change?

9. Optional DSA Revision (30 Minutes)

Redo with a 10-minute limit each: Kth Largest Element (min-heap of size k), Find Median from Data Stream (two heaps), Koko Eating Bananas (binary search on the answer) and Implement Trie. Write the pattern name next to each.

10. End-of-Day Checklist

- [ ] An order placed through the gateway reaches CONFIRMED or CANCELLED through Kafka events.
- [ ] Every consumer uses an inbox (or a conditional state change), and duplicate delivery is proven harmless by a test.
- [ ] Stock reads are cached, order creation is rate limited, and the API survives Redis being down.
- [ ] Unit, slice and Testcontainers integration tests pass with one command.
- [ ] I ran the seven failure drills and wrote down what happened.
- [ ] Everything is committed to GitHub with clear messages.
- [ ] I can answer the ten talking points without notes.

Tomorrow (Week 4, Day 6, weekend): project polish. Add Resilience4j to the inventory call, expose Actuator and Micrometer metrics, write the README with an architecture diagram, and push the final version to GitHub.